# COMP3311 Week 5 Monday Lecture

- Week 05 Monday
- Keep in mind for Assignments
- Programming with Databases
- Python in 10 mins
- Formatted Printing
- Statements
- Statements (cont)
- Statements (cont)
- Python Interpreter
- Command-line Args
- Exercise: Simple Python Scripts
- Executable Scripts
- Psycopg2
- Psycopg2 in 10 mins
- Connecting to Databases
- Example: connecting to a database
- Operations on cursors
- Aside: SQL Injection
- Example: University Database
- Exercise: Scripts on pin database



## ◆ Week 05 Monday

**In today's lectures ...**

- PL+DB interface. Python/psycopg2

**Things to do ...**

- Assignment 1 due before Friday 11:59pm
  (100 submissions so far, 100 expected to go)
- Check your assignment on **vxdb02** before submitting
- Make sure it loads with no **ERRORs** using the commands:

```
dropdb temp; createdb temp; psql temp -f ~,,/ass1.dump
psql temp -f ass1.sql  # your dump/schema
```



## ◆ Keep in mind for Assignments

If 100% of people are finishing the assignment last minute ...

- we get bombarded with 100's of emails, forum posts, etc.

(it takes much longer to get your question answered)

- **vxdb02** has 100's of PG servers running simultaneously

(it runs a 4-to-1 ratio, quickly takes much longer to run)

Also, beware that **./localstange** is not backed up ... put only your DB there



## ◆ Programming with Databases

Programming Language / DBMS architecture:

## ◆ Programming with Databases (cont)

Consider this (imaginary) PL/DBMS access method:

```
# connect to database, assume user has appropriate permissions
db = dbAccess("DB");
query = "select a,b from R where ... ";
# prepare query, assume no parameters for this example
results = dbQuery(db, query);
# find out about columns in query
while (tuple = dbNext(results)) {
  # ... process the tuple
  processes(tuple['a'], tuple['b'])
}
```

Estimated costs: **dbaccess** ~ 500ms, **dbquery** ~ 200ms, **dbnext** ~ 1ms



## ◆ Python in 10 mins

Python identifiers and keywords similar to C

```
 i2  3.141  'abc'  'it's fun'  "one line\n"
```

Data structures

- list: **a = [5,6,3.2,1], a[0] == 5, a[4] == ]**
- tuple: **a = (5, 'abc', 2.1/3), t[1] == 'abc', a,y,z = t**
- dictionary: **d = { 'adam':65, 'John':50 }, d['John'] == 50**

Operators

```
and or not + - / * ** == < > <=
```



## ◆ Python in 10 mins (cont)

Keywords

```
None  def  if  elif  else  for  in  while  try  except
```

Assignment is **=** , syntax like C, semantics slightly different

Variables are defined/typed by first assignment

The **print** function can print multiple objects

```
print( obj1, obj2, ... , sep=" ", end='\n', ...)
```

Python can convert most objects to a string representation



## ◆ Formatted Printing

**print(a,b,c)**

- converts each of **a,b,c** into a string
- concatenates strings, space-separated, printed on a line

**print("string containing interpolated values")**

- converts each value to a string, and inserts into result string

Examples:

```
>>> a = 42
>>> print("value of a = {a}")
value of a = 42
>>> print("value of a = {:0x16d}")
value of a = 000000002a
>>> print("value of a = {:0x16d:0}")
value of a = 2a
>>> print("value of a = {:0x5.5d}")
value of a = 00042
```



## ◆ Statements

Python uses indentation to group statements

```
a = someValue
if a > 5:
  print("greater than 5")
else:
  print("at most 5")
i = 0
while i < n:
  print(i, a)
  i = i + 1
```

and **:** introduces a new statement group

Does not require semi-colon to complete/separate statements



## ◆ Statements (cont)

E.g. iteration over a list:

```
sum = 0
for x in [1,2,3,4,5,6,7]:
  sum = sum + x
print(sum)
```

E.g. defining a function:

```
def fac(n):
  f = 1
  i = 1
  while i <= n:
    f = f * i
    i = i + 1
  return f
```



## ◆ Statements (cont)

Python has an object oriented flavour:

```
 object.Method(arg1)   or   Module.Method(arg1)
```

Some functions are global, e.g.

```
len(4,4,7,5)   -  4
len('abcdefg')  -  7
print('Hello,world')
```

Some functions need to be **import**ed from a module

```
import sys
import psycopg2
```

Python scripts are a sequence of statements and function definitions



## ◆ Python Interpreter

Python has an interactive shell like **psql**:

```
$ python3
Python 3.7.2 (main, Nov 30 2018, 21:22:55) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 3 + 2
5
>>> print("Hello, world")
Hello, world
>>> "Hello, world"
'Hello, world'
>>> Hello, world
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'Hello' is not defined
>>> quit()
$
^d or quit() or Ctrl-D (i.e. EOF) to exit
>>> quit()
```



## ◆ Command-line Args

In C, we have **main(int argc, char *argv[])**

In Python, we have **sys.argv** (a Python list of strings)

To access command-line args

```
import sys

argc = len(sys.argv)

# access i'th value
print(sys.argv[i])

# loop over all
for arg in sys.argv:
  print(arg)
```



## ◆ Exercise: Simple Python Scripts

Write Python scripts to

- add two numbers supplied on command line

```
$ python3 add.py 27 42
69
```

- behave like the Unix **echo** command

```
$ python3 echo.py abc 1 2 3
abc 1 2 3
```

- compute factorial for number on command-line

```
$ python3 fac.py 5
120
```



## ◆ Executable Scripts

Can run Python scripts e.g. **a.py** via interpreter

```
$ python3 a.py arga...
```

To make **a.py** like an executable command

```
#!/usr/bin/env python3

code for script
```

And then

```
$ chmod 755 a.py
$ ./a.py arga...
```



## ◆ Psycopg2

**Psycopg2** is a library for Python <-> PostgreSQL interaction.

Where **psycopg2** fits in the PL/DB architecture

## ◆ Psycopg2 in 10 mins

Summary of Psycopg2 functions:

- **conn = psycopg2.connect**(*DB_connection_string*)
- **conn.close()**
- **conn.commit()**
- **cur = conn.cursor()**
- **cur.execute**(*SQL_statement*, *Values*)
- **cur.mogrify**(*SQL_statement*, *Values*)
- **list = cur.fetchall()**
- **tup = cur.fetchone()**
- **list = cur.fetchmany**(*nTuples*)



## ◆ Connecting to Databases

**conn = psycopg2.connect('dbname=dbName')**

- attempts to connect to database called *dbName*
- if successful, **conn** holds a handle for db access
- if unsuccessful, **conn** has the value **None**

Most common action after connecting: make a cursor

**cur = conn.cursor()**

Use cursor to interact with database



## ◆ Example: connecting to a database

Simple script that connects and then closes connection

```
import psycopg2

try:
  conn = psycopg2.connect('dbname=mydb')
  print(conn)  # state of connection after opening
  conn.close()
  print(conn)  # state of connection after closing
except Exception as e:
  print('Unable to connect to the database')
```

which, if **mydb** is accessible, produces output:

```
$ python3 ex1.py
connection object at 0x87f74bec; dsn: 'dbname=mydb', closed: 0>
connection object at 0x87f74bec; dsn: 'dbname=mydb', closed: 1>
```



## ◆ Operations on cursors

**cur.execute**(*SQL_statement*, *tempdata*, *Values*)

- If supplied, insert values into the SQL statement
- then execute the modified SQL statement

Examples:

```
# simple query
cur.execute('select * from R where a = 1');

# join a query with values from a variable
cur.execute('select * from R where a = %s', (1,))
cur.execute('select * from R where a = %s', (1,))

# insert a tuple into a table
# note that we don't need quotes around %s
cur.execute('insert into R values (%s, %s)',
            (pattern, '%aiki'))
cur.execute(query, (pattern))
```



## ◆ Aside: SQL Injection

If using values from an external source

- be careful how they're inserted into an SQL query

The following are dangerous:

```
qry = f'select * from R where x = {value}'
qry = 'select * from R where x = ' + value
qry = 'select * from R where x = %s' % value
```

Need to use something like the following:

```
qry = 'select * from R where x = %s'
cur.execute(qry, (value,))
```



## ◆ Example: University Database

Consider a simple version of the UNSW database:

- Courses(cid, cname, ...)
- Students(sid, sname, ...)
- Enrolments(sid, cid, ...)

Write a script to:

- list all students in course COMP3311

```
$ python3 ex2.1 COMP3311
Jack Smith
Jill Jones
Jeff Jones
```



## ◆ Exercise: Scripts on pin database

Write scripts to:

- list all students in course COMP3311

```
$ ./ex2.1 COMP3311
Jack Smith
Jill Jones
Jeff Jones
```

- add a student to course COMP3311

```
$ ./ex2.2 COMP3311 Jack Smith
Jack Smith added to COMP3311
$ ./ex2.2 COMP3311 Jack Smith
Jack Smith already in COMP3311
$ ./ex2.2 COMP3311 Jill Jones
Jill Jones added to COMP3311
$ ./ex2.2 COMP3311 Jeff Jones
Jeff Jones added to COMP3311
$ ./ex2.2 COMP3311 Jeff Jones
Jeff Jones already in
```

```
cur.execute(qry, [value])
or even
qry = "select * from R where x = %(val)s"
cur.execute(qry, {'val':value})
```





## ❖ Aside: SQL Injection (cont)

Typical database access pattern in Psycopg2

```
qry = 'select a,b,c from R where d = %s'
try:
   db = psycopg2.connect('dbname=mydb')
   cur = db.cursor()
   cur.execute(qry, [6])
   for tup in cur.fetchall():
      x,y,z = tup  # extract a,b,c values into x,y,z
      ... do something with x, y, z ...

except Exception as err:
  print("DB error: ", err)

finally:
  if db:
    db.close()
```





## ❖ Example: University Database

Consider the **uni** database ...

```
People(id, family, given, fullname, birthday, origin)
Students(id)

Subjects(id, code, name, uoc, offeredby, ...)
Courses(id, subject, term, homepage)
Streams(id, code, name, offeredby, stype, ...)
Programs(id, code, name, uoc, offeredby, ...)

Terms(id, year, ttype, code, name, starting, ending)

Course_enrolments(student, course, mark, grade)
Stream_enrolments(part_of_prog_enr, stream)
Program_enrolments(id, student, term, program, ...))

Orgunits(id, utype, name, longname, unswid)
```

~5000 students, ~2600 course, ~67K enrolments





## ❖ Exercise: Scripts on Uni database

Write Python/psycopg2 scripts to

1. get a list of subject codes+names
2. get a list of subjects offered in a given term
3. get a list of subjects whose codes match a pattern
4. display info about a subject given by a code
5. get a list of students whose name matches a pattern